

CISP 400

Object Oriented Programming in C++

Dr. Fowler



F O L S O M L A K E C O L L E G E
EL DORADO CENTER | RANCHO CORDOVA CENTER

Last Time

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Theme for Today

“Method.”

Chapter 7

Object Oriented Programming

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

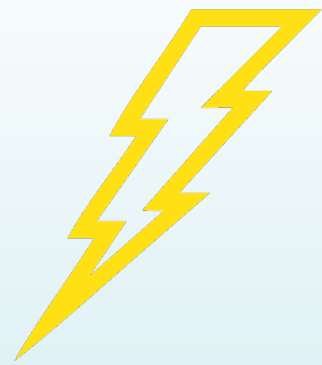
7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

A large, stylized yellow lightning bolt graphic is positioned diagonally across the center of the slide. It has a jagged, multi-segmented path, starting from the top right and ending at the bottom left. The bolt is a solid yellow color with a slight gradient, giving it a three-dimensional appearance.

Lightning Review



Lightning Review Questions

Great time to get Peer Review Partners

- What is a dependency relationship?
- What other questions to you have for me?

Homework 2 Preflight

Exercise

Debrief

Programming Exercise

Coding Exercise

Create a Date class with private member ints day, month, and year. Add a print public member function to display the date. Use constructors to initialize to specified values or default values if nothing is specified. Do not allow the class to hold invalid dates.

Write a test rig to exercise this class.

```

1  Date2.cpp
2  #include<iostream>
3  using namespace std;
4  class Date {
5      private:
6          int mo;
7          int day;
8          int year;
9      public:
10         Date(int = 1, int = 1, int = 1);
11         void PrintDate();
12     };
13     Date::Date(int m, int d, int y) {
14         mo = m;
15         day = d;
16         year = y;
17         if (mo > 12) mo = 99; // 99 is error SetDate
18         // other checking goes here
19     }
20     void Date::PrintDate() {
21         cout << mo << "/" << day << "/" << year << endl;
22     }
23
24     int main() {
25         Date date1, date2(10,15,2009);
26         Date date3(21,25,2017);
27         cout << "Using default arguments: ";
28         date1.PrintDate();
29         cout << "Using passed arguments: ";
30         date2.PrintDate();
31         cout << "Error checking: ";
32         date3.PrintDate();
33     }

```

```

clang version 7.0.0-3~ubuntu0.18.04.1
➤ clang++-7 -pthread -o main main.cpp
➤ ./main
Using default arguments: 1/1/1
Using passed arguments: 10/15/2009
Error checking: 99/25/2017

```

10.12 Selecting Members of Objects

What do we do when we need to access pointers
inside objects?


```
1  //AccessPointers.cpp
2  #include <iostream>
3  using namespace std;
4
5  struct GradeInfo {
6      int* pTestScores = nullptr;
7      double average = 0;
8  };
9
10 int main() {
11     GradeInfo s1;
12     s1.pTestScores = new int{93};
13     GradeInfo* pS1 = &s1;
14
15     // Your task: Access the variables w and w/o a ptr
16     cout << "1: " << s1.average << endl;
17     cout << "2: " << *s1.pTestScores << endl;
18     cout << "3: Access average with the ptr variable\n";
19     cout << "4: Access pTestScores with the ptr variable\n";
20
```

```
22  cout << "3: " << *pS1.average << endl;
23  //FAIL!
24  // Order of precedence problems again. Line 21 parsed as
25  /*(pS1.average) and treated as 1 pointer. But this doesn't have a
    member variable average, only pS1 does.
```

ERROR!

/tmp/G3kKsLNy5/main.cpp: In function 'int main()':

/tmp/G3kKsLNy5/main.cpp:22:23: error: request for member 'average' in 'pS1',
which is of pointer type 'GradeInfo*' (maybe you meant to use '->' ?)

```
22 | cout << "3: " << *pS1.average << endl;
```

```
|
```

```
^~~~~~
```

=== Code Exited With Errors ===

```
26  cout << "3: " << (*pS1).average << endl;
```

```
1: 0  
2: 93  
3: Access average with the ptr variable  
4: Access pTestScores with the ptr variable  
3: 0
```

Use the parentheses to make sure the * binds first.

Turn our attention to two pointers.

```
28  cout << "4: " << *pS1.pTestScores << endl;
29  //FAIL - you would think it would work with 2 ptrs!
30  // But we need to dereference both pointers
```

ERROR!

/tmp/uu1NjZZ2yK/main.cpp: In function 'int main()':

/tmp/uu1NjZZ2yK/main.cpp:28:23: error: request for member 'pTestScores' in
 'pS1', which is of pointer type 'GradeInfo*' (maybe you meant to use '->'
 ?)

```
28 | cout << "4: " << *pS1.pTestScores << endl;
   |                   ^~~~~~
```

=== Code Exited With Errors ===

```
32  cout << "4: " << pS1.*pTestScores << endl;
33  // FAIL - must dereference both
```

ERROR!

/tmp/HhOBWlvBSt/main.cpp: In function 'int main()':

/tmp/HhOBWlvBSt/main.cpp:32:23: error: 'pTestScores' was not declared in this scope

```
32 | cout << "4: " << pS1.*pTestScores << endl;
   |                      ^~~~~~
```

=== Code Exited With Errors ===

```
35  cout << "4: " << *pS1.*pTestScores << endl;
36  // FAIL - makes no sense
```

ERROR!

/tmp/LF8029s0s0/main.cpp: In function 'int main()':

/tmp/LF8029s0s0/main.cpp:35:24: error: 'pTestScores' was not declared in this scope

```
35 | cout << "4: " << *pS1.*pTestScores << endl;
    |                      ^~~~~~
```

=== Code Exited With Errors ===

Parentheses to the rescue again.

```
38  cout << "4: " << *(*pS1).pTestScores << endl;  
39  // Second * binds to pS1, then first * binds to pTestScores  
40  // Now we are dereferenceing both ptr variables
```

```
1: 0  
2: 93  
3: Access average with the ptr variable  
4: Access pTestScores with the ptr variable  
3: 0  
4: 93
```


The structure pointer operator can also help us.

```
42 // Also use the structure pointer operator (->)
43 cout << "4a: " << *pS1->pTestScores << endl;
44 // or to make it explicit
45 cout << "4b: " << *(pS1->pTestScores) << endl;
46
```

```
1: 0
2: 93
3: Access average with the ptr variable
4: Access pTestScores with the ptr variable
3: 0
4: 93
4a: 93
4b: 93
```

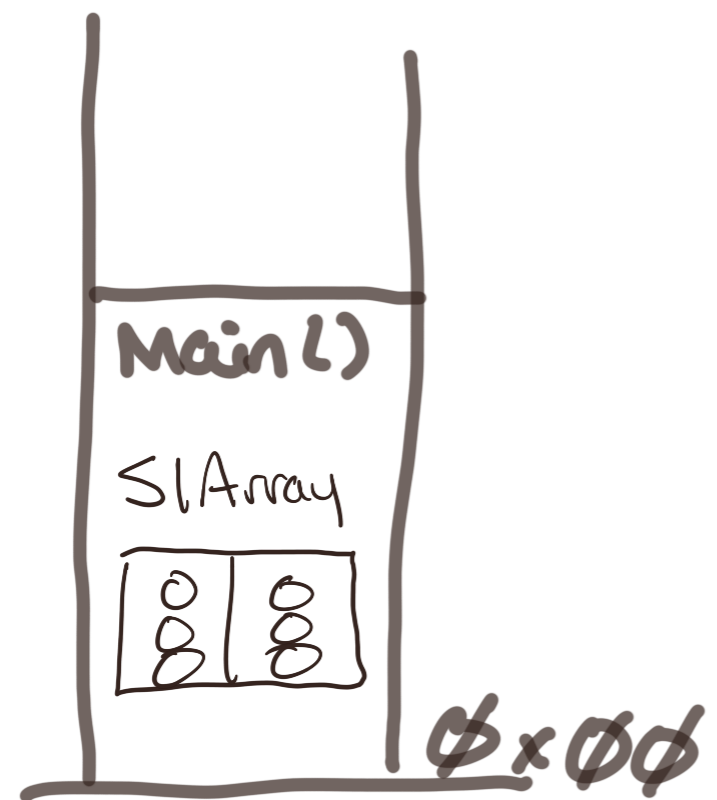
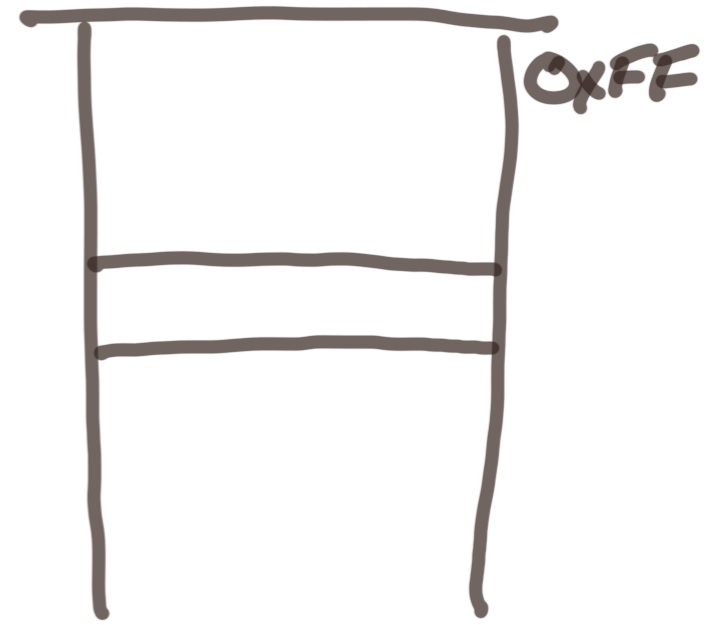
=== Code Execution Successful ===


```
1  #include <iostream>
2  using namespace std;
3
4  struct GradeInfo {
5      string name = "Sue";
6      int* pTestScores;
7      double average = 0;
8  };
9
10 int main() {
11     GradeInfo s1;
12     GradeInfo* pS1 = &s1;
13
14     // do this way because just 1 var is a ptr
15     *s1.pTestScores = 93;
16     cout << *s1.pTestScores << endl;
17
18     // do this way because both var are ptr
19     *pS1->pTestScores = 88;
20     *(*pS1).pTestScores = 99;
21 }
```

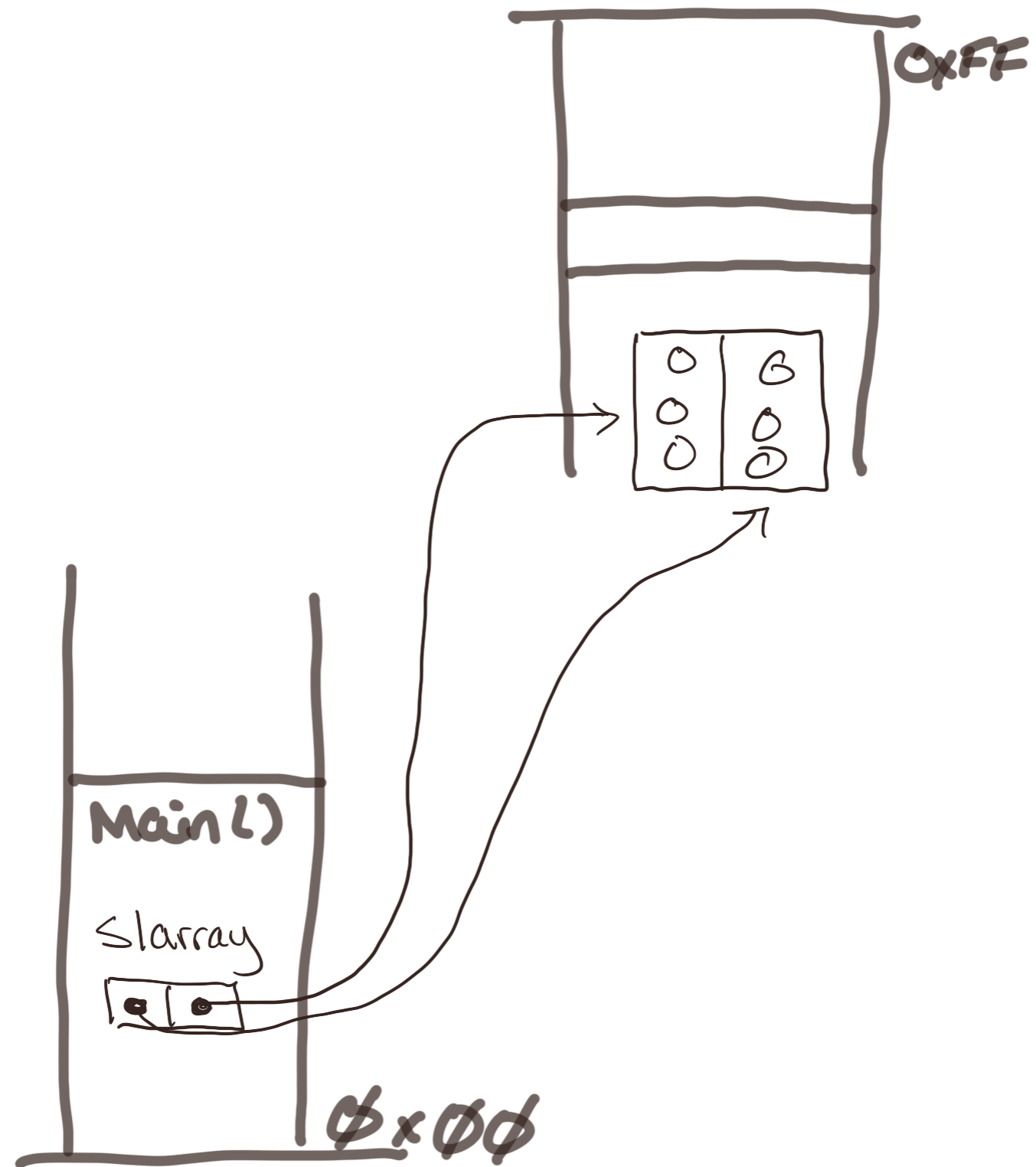
```
./main
93

```

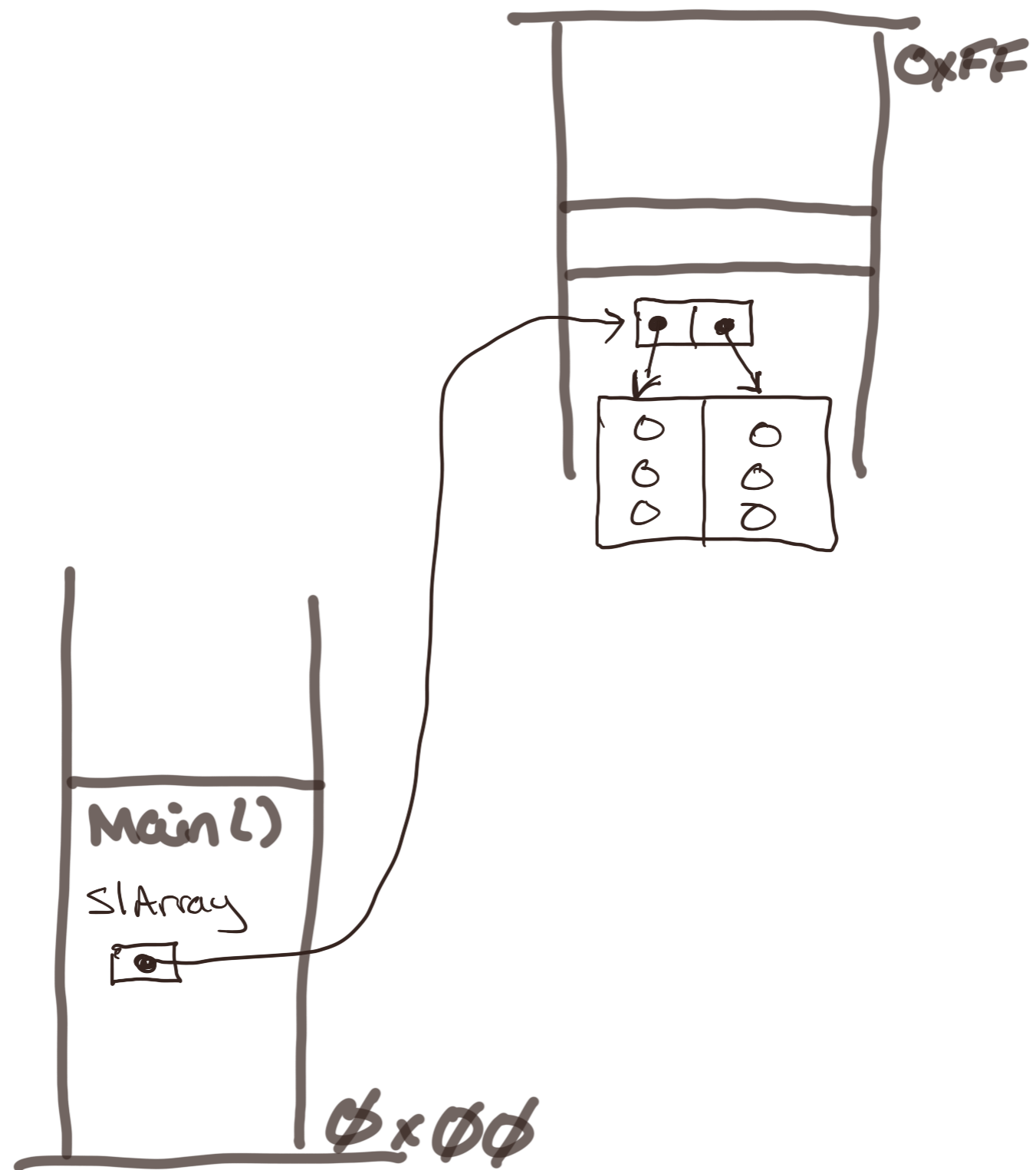
8.13 Arrays of Objects



```
1 // ds1a0.cpp -- demo of array of stoplight class
2 #include <iostream>
3 using namespace std;
4
5 class StopLight {
6 public:
7     enum Light {RED, YELLOW, GREEN};
8     Light steadyLight;
9     StopLight(){ steadyLight = YELLOW; };
10    StopLight(Light lightColor){ steadyLight = lightColor;};
11    void Cycle();
12 };
13 void StopLight::Cycle() {
14     switch (steadyLight){
15         case RED: cout << "Red\n"; break;
16         case YELLOW: cout << "Yellow\n"; break;
17         case GREEN: cout << "Green\n"; break;
18     }
19 }
20
21 int main(){
22     StopLight slArray[2];
23     slArray[0].Cycle();
24     slArray[1].Cycle();
25     cout << "\nCreate objects.\n\n";
26
27     slArray[0] = StopLight(StopLight::RED);
28     slArray[1] = StopLight(StopLight::YELLOW);
29
30     slArray[0].Cycle();
31     slArray[1].Cycle();
32
33     return 0;
34 }
```



```
1 // ds1a1.cpp -- demo of array of stoplight class pointers
2 #include <iostream>
3 using namespace std;
4
5 class StopLight {
6 public:
7     enum Light {RED, YELLOW, GREEN};
8     Light steadyLight;
9     StopLight(){ steadyLight = YELLOW; };
10    StopLight(Light lightColor){ steadyLight = lightColor;};
11    void Cycle();
12 };
13 void StopLight::Cycle() {
14     switch (steadyLight){
15     case RED: cout << "Red\n"; break;
16     case YELLOW: cout << "Yellow\n"; break;
17     case GREEN: cout << "Green\n"; break;
18     }
19 }
20
21 int main(){
22     StopLight* slArray[2];
23     // slArray[0]->Cycle(); // warning w/ seg fault if run
24     slArray[0] = new StopLight(StopLight::RED);
25     slArray[1] = new StopLight(StopLight::YELLOW);
26
27     slArray[0]->Cycle();
28     slArray[1]->Cycle();
29
30
31     return 0;
32 }
```




```
1  // ds1a2.cpp -- dynamic array of stoplight class pointers
2  #include <iostream>
3  using namespace std;
4
5  class StopLight {
6  public:
7      enum Light {RED, YELLOW, GREEN};
8      Light steadyLight;
9      StopLight(){ steadyLight = YELLOW; };
10     StopLight(Light lightColor){ steadyLight = lightColor;};
11     void Cycle();
12 };
13 void StopLight::Cycle() {
14     switch (steadyLight){
15     case RED: cout << "Red\n"; break;
16     case YELLOW: cout << "Yellow\n"; break;
17     case GREEN: cout << "Green\n"; break;
18     }
19 }
20
21 int main(){
22     StopLight** slArray = new StopLight*[2];
23     // slArray[0]->Cycle();    // warning w/ seg fault if run
24     slArray[0] = new StopLight(StopLight::RED);
25     slArray[1] = new StopLight(StopLight::YELLOW);
26
27     slArray[0]->Cycle();
28     slArray[1]->Cycle();
29
30
31     return 0;
32 }
```


Exercise

- Design a library system with three classes: book, author, and publisher. An author writes books. A publisher publishes books. A book must know who wrote it and who published it. Add any other variables you see fit.
- Create a program that: 1) defines three classes with the correct data members and dependency relationships, 2) create at least two authors, two publishers, and three books, and 3) print each book title, author, and publisher using dependency relationships. The print method should be in the book class.

```

#include <iostream>
#include <string>
using namespace std;

class Author {
public:
    string name;
    Author(string n) : name(n) {}
};

class Publisher {
public:
    string name;
    Publisher(string n) : name(n) {}
};

class Book {
public:
    string title;
    Author* author;    // dependency on Author
    Publisher* publisher; // dependency on Publisher

    Book(string t, Author* a, Publisher* p)
        : title(t), author(a), publisher(p) {}

    void printInfo() {
        cout << "Book: " << title
              << ", Author: " << author->name
              << ", Publisher: " << publisher->name
              << endl;
    }
};

int main() {
    Author a1("J.K. Rowling");
    Author a2("George Orwell");

    Publisher p1("Bloomsbury");
    Publisher p2("Secker & Warburg");

    Book b1("Harry Potter", &a1, &p1);
    Book b2("1984", &a2, &p2);
    Book b3("Animal Farm", &a2, &p2);

    b1.printInfo();
    b2.printInfo();
    b3.printInfo();
}

```

```

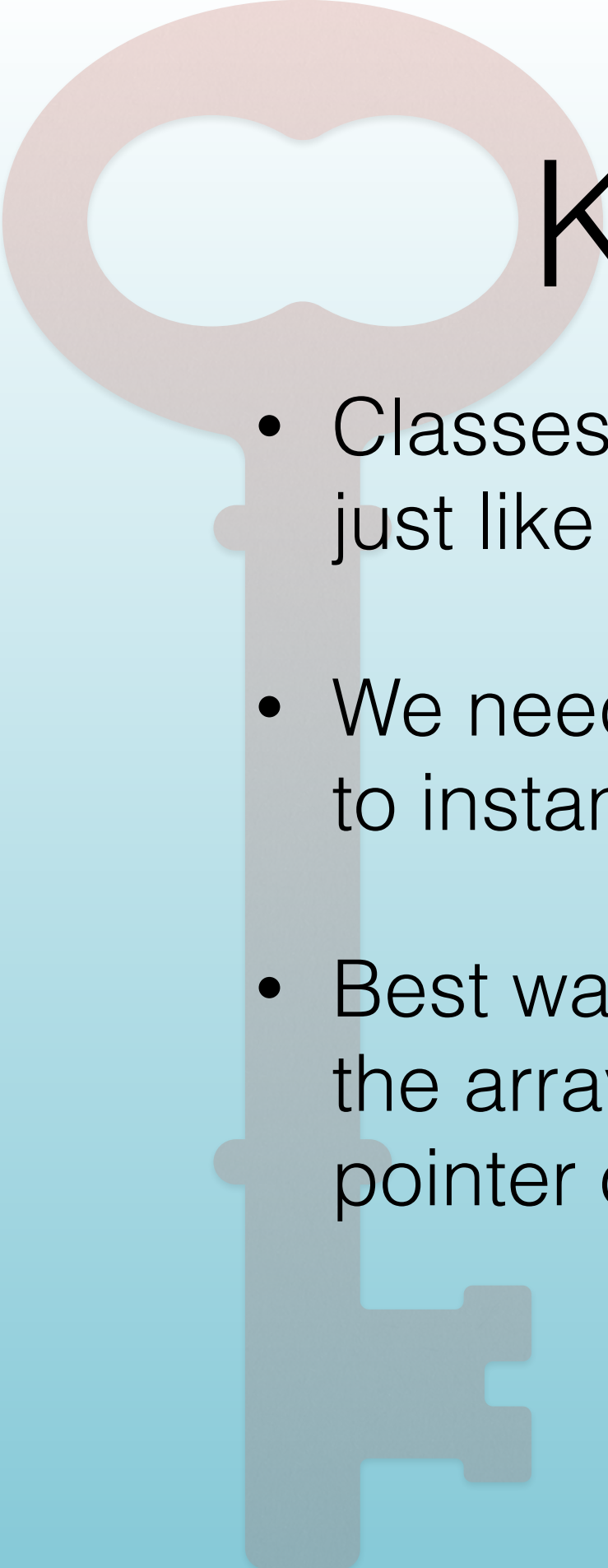
Book: Harry Potter, Author: J.K. Rowling, Publisher: Bloomsbury
Book: 1984, Author: George Orwell, Publisher: Secker & Warburg
Book: Animal Farm, Author: George Orwell, Publisher: Secker &
      Warburg

```

```

=== Code Execution Successful ===

```



Key Takeaway

- Classes are types & we can pass them to methods just like any other type.
- We need to give some thought as to where we want to instantiate our objects - stack or heap?
- Best way to use arrays of objects is to keep both the array and the objects on the heap and the array pointer on the stack.



What's this section
about?

Theme for Today

“Passing Objects.”

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

What's Next?

Agenda

Ch 12 - Strings

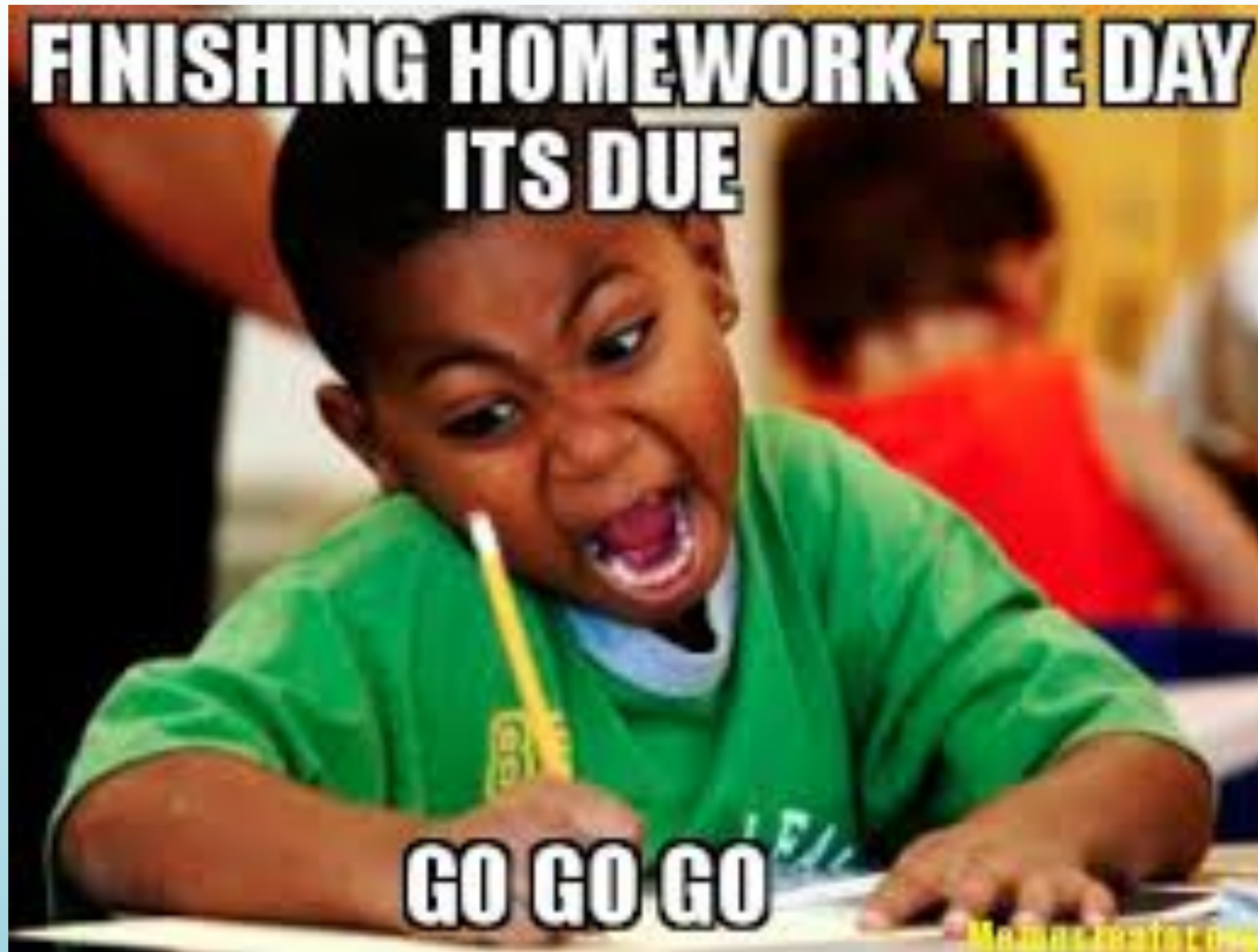
12.1 C-Strings

12.2 Library Functions for Working with C- Strings

12.3 Conversions between Numbers and Strings

12.4 Writing Your Own C-String Handling Functions

12.5 More about the C++ String Class



Homework 2 due Sunday.

Dueling Questions

Exercise

Exercise Goals

- Develop your critical thinking skills.
- Increase topic mastery
- Construct Knowledge
- Improve Workplace Skills (teamwork, feedback)
- Improve Interpersonal Skills

Dueling Questions

- Pair up, get some paper out.
- You may only communicate with your partner by asking a question about **a topic in this chapter**.
- Do NOT answer, discuss or debate the questions.
- Your partner will answer you with another question. Change any statements to a question.
- Record all questions.

Dueling Questions

- Change any closed questions to open questions.
- Decide as a group on the most interesting question.

(1) CppCon 2014 Lightning Talks - Michael VanLoon Anatomy of a Smart Pointer - YouTube

RAII

- Resource Acquisition is Initialization
 - Encapsulate resources in classes (constructor acquires, destructor releases).
 - Use resource via local instances of class.
 - Automatically free resource when object goes out of scope.

Homework 2 Peer Review

Exercise

Hw Debrief

Increases topic mastery.
Construct knowledge.

Process

- Pair up with Group B
- Review each other's homework.
- Complete the Review Sheet.
- Exchange sheets.
- Reflect and comment on the feedback you received.
- Turn it in.

7.9 Passing Objects to Functions

Ways to pass objects to functions

- Pass by reference variable
- Pass by reference (pointer)
- Pass by value
- Pass by constant reference

- Download and execute 7-9mod.cpp

- Get some paper - not turn in.
- Take a few seconds to collect your thoughts.
- What are the advantages and disadvantages of the 4 methods?

Think-Pair-Share

Exercise

Think-Pair-Share

Promotes discussion and thought.

Answer any outstanding issues from last class.

Think

- Get some paper - not turn in.
- Take a few seconds to collect your thoughts.
- What are the advantages and disadvantages of the 4 methods?

Pair

- Pair up
- Discuss your answers

Share

- Share with the class

Debrief

What worked about this exercise?

What could be improved about this exercise?

Bottom Line

- Classes are custom **types**.
- We can pass them around, just like any other type.